# JavaScript Random numbers and Control Statements

Comprehensive Guide with Examples

# JavaScript Math.random()

- The Math.random() function is used to generate a **pseudo-random number** between 0 (inclusive) and 1 (exclusive).
- This means it will always return a floating-point number greater than or equal to 0 but less than 1.

# How Math.random() Works

- **Range:**
  - Returns values in the range [0, 1) (includes 0 but excludes 1).
- **Pseudo-Random:**
  - The numbers generated are deterministic and based on an algorithm, so they are not truly random.
- Example

```
let randomNumber = Math.random();
console.log(randomNumber); // Example: 0.48257934957023
```

# Generating a Random Number in a Range

- To generate random numbers in a custom range, use the following formula:

```
let randomInRange = Math.random() * (max - min) + min;
```

- Example: Random number between 5 and 10

```
let min = 5;
let max = 10;
let randomInRange = Math.random() * (max - min) + min;
console.log(randomInRange); // Example: 7.345823
```

# Generating a Random Integer

- To get an integer instead of a floating-point number, combine Math.random() with Math.floor() or Math.ceil().
- **Example: Random integer between 1 and 100**

```
let randomInteger = Math.floor(Math.random() * 100) + 1;
console.log(randomInteger); // Example: 42
```

# Simulating a Dice Roll

- A dice has values from 1 to 6. Use the formula Math.floor(Math.random() * 6) + 1 to simulate a dice roll.

```
let diceRoll = Math.floor(Math.random() * 6) + 1;
console.log(`You rolled: ${diceRoll}`); // Example: You rolled: 4
```

# Practical Uses

- Games (e.g., dice rolls, random item generation)
- Shuffling elements
- Generating random test data
- Creating random keys or tokens

# Using if-Else Conditionals

- The if-else statement allows you to execute different blocks of code based on conditions.
- if-else statements control the flow based on conditions.
- Syntax:

```
if (condition) {
  // code if true
} else {
  // code if false
}
```

# Example

```
let age = 18;
if (age >= 18) {
  console.log('Eligible to vote');
} else {
  console.log('Not eligible to vote');
}
```

# If-Else: Real-World Example

- Example: Checking driving eligibility

```
let age = 16;
if (age >= 18) {
  console.log('You can drive.');
} else {
  console.log('You are too young to drive.');
}
```

# If-Else: Multiple Conditions

- Nested if-else checks multiple conditions sequentially.
- Example: Assigning grades

```
let score = 85;
if (score >= 90) {
  console.log('Grade: A');
} else if (score >= 80) {
  console.log('Grade: B');
} else {
  console.log('Grade: C');
}
```

# Comparators and Equality

- JavaScript supports various comparison operators:
- `==` (loose equality)
- `===` (strict equality)
- `!=` and `!==`
- `<`, `<=`, `>`, `>=`
- Example:

```
console.log(5 == '5');   // true
console.log(5 === '5');  // false
console.log(10 > 5);     // true
```

# Strict vs Loose Equality

- `==` compares values regardless of type.
- `===` compares both value and type.
- Example:
- Loose Equality (==)

```
console.log(5 == '5'); // true
```

- Strict Equality (===)

```
console.log(5 === '5'); // false
```

# Combining Comparators

- Use logical operators to combine conditions:
- `&&` (AND): Both conditions must be true.
- `||` (OR): At least one condition must be true.
- `!` (NOT): Negates the condition.
- Example: Voting eligibility

```
let age = 20;
let citizenship = 'USA';
if (age >= 18 && citizenship === 'USA') {
  console.log('You can vote.');
} else {
  console.log('You cannot vote.');
}
```

# Challenge

- BMI Calculator Advanced (IF/ELSE)
- Leap Year exercises